

Système de reconnaissance faciale événementielle

Analyse technique & architecture cible — Note de cadrage CTO

Source : [Besoin.odt](#) — Besoin fonctionnel « indexation & recherche de personnes dans une base d'événements »
Date : 19 juillet 2026 • Auteur : Direction Technique • Statut : proposition d'architecture — v1.0

Synthèse en une phrase. Le besoin exprimé est un moteur de *recherche par similarité biométrique* : ce n'est pas un problème de vision par ordinateur (les modèles sont des commodités matures), c'est un problème de **pipeline de données, de recherche vectorielle à grande échelle, et surtout de conformité juridique sur données biométriques**. Le risque projet n'est pas technique, il est réglementaire et qualitatif (taux de faux positifs).

0. Sommaire

- | | |
|--|--|
| 1. Résumé exécutif & recommandation | 10. Qualité : seuils, métriques, biais |
| 2. Analyse du besoin et zones d'ombre | 11. Dimensionnement & performance |
| 3. Enjeu bloquant : conformité biométrique | 12. Contrat d'API |
| 4. Principes d'architecture | 13. Sécurité & exploitation |
| 5. Architecture cible (vue composants) | 14. Coûts & build vs buy |
| 6. Pipeline d'indexation (détail) | 15. Trajectoire de mise en œuvre |
| 7. Pipeline de recherche (détail) | 16. Registre des risques |
| 8. Modèle de données | 17. Décisions d'architecture (ADR) |
| 9. Choix technologiques & alternatives | |

1. Résumé exécutif & recommandation

Le système demandé se décompose en deux chaînes fonctionnelles asymétriques :

- **Chaîne d'écriture (indexation)** — traitement *batch* et asynchrone, tolérant à la latence, coûteux en calcul. Elle parcourt les images des événements, détecte les visages, produit des vecteurs d'empreinte (*embeddings*) et les persiste.
- **Chaîne de lecture (recherche)** — traitement *temps réel*, exigeant en latence (< 1 s perçue), peu coûteux unitairement. Elle transforme une photo requête en vecteur et interroge un index de similarité.

Ces deux chaînes ont des profils de charge opposés et **doivent être déployées et dimensionnées séparément**. C'est la décision structurante n°1.

Recommandation

Stack cible recommandée : détection [SCRFD](#) + embedding [ArcFace](#) / [InsightFace](#) ([buffalo_l](#), 512-d) , exécutés en ONNX Runtime dans un *worker* Python isolé ; persistance [PostgreSQL](#) + [pgvector](#) (index HNSW) ; orchestration par file de messages ([Redis Streams](#) ou [RabbitMQ](#)) ; API de recherche en [FastAPI](#) . Modèles auto-hébergés — **pas de service cloud tiers** pour la biométrie.

Pourquoi : une seule base à opérer (PostgreSQL contient déjà les événements — pas de synchronisation à deux systèmes), souveraineté des données biométriques, coût marginal nul par requête, et une bascule vers Qdrant/Milvus reste possible sans réécrire le domaine si le volume dépasse ~10 M de vecteurs.

512

dimensions par empreinte
faciale (float32 → 2 Ko)

~0,38

seuil de distance cosinus cible
(à calibrer)

< 300 ms

latence de recherche visée
(p95, hors réseau)

~40 img/s

débit d'indexation par GPU T4
(est.)

Effort estimé

Lot	Charge	Contenu
Lot 0 — Cadrage juridique	2 sem.	Base légale, consentement, DPIA, politique de rétention. Bloquant.
Lot 1 — Socle d'indexation	4–5 sem.	Workers, détection, embedding, schéma, reprise sur erreur, backfill.
Lot 2 — Recherche	3 sem.	Index HNSW, API, agrégation par événement, calibration du seuil.
Lot 3 — Qualité & industrialisation	3–4 sem.	Jeu d'évaluation, observabilité, purge RGPD, back-office de modération.
Total ≈ 12–14 semaines		Équipe : 1 ingénieur ML/CV, 1 ingénieur backend, 0,5 DevOps, 0,3 juridique/DPO.

2. Analyse du besoin et zones d'ombre

2.1 Ce que le document exprime

Élément	Lecture technique
Événement = description, date, adresse, 1..n images	Entité racine existante. La reconnaissance faciale est une couche d'enrichissement , pas une refonte : le modèle de données existant n'est pas modifié, il est étendu.
« parcourir automatiquement l'ensemble des événements »	Impose un backfill initial sur l'historique complet + un traitement incrémental à chaque nouvel événement. Deux modes d'exécution, un seul pipeline.
« empreintes biométriques ou vecteurs de reconnaissance faciale »	Le besoin cible bien un <i>embedding</i> vectoriel. Confirme la nécessité d'un moteur de recherche par similarité (ANN), pas d'une recherche SQL classique.
« retourner la liste des événements »	Le résultat est agrégé au niveau événement , pas au niveau visage. Impose une étape de regroupement + scoring d'événement en sortie de l'ANN.

2.2 Zones d'ombre à arbitrer **À TRANCHER**

Ces sept points changent l'architecture. Ils doivent être tranchés avant le Lot 1.

#	Question	Impact architectural
1	Volumétrie cible ? Nombre d'événements, d'images/événement, de visages/image.	Détermine pgvector vs Qdrant, CPU vs GPU, mono-nœud vs distribué. <i>Hypothèse de travail retenue : 100 k événements × 8 images × 4 visages ≈ 3,2 M vecteurs.</i>
2	Identités connues ou anonymes ? Dispose-t-on d'un référentiel de personnes nommées ?	Si oui → problème d' <i>identification</i> (1:N contre galerie labellisée). Si non → clustering non supervisé des visages en identités latentes. Change tout le modèle de données.
3	Qui a le droit de rechercher ? Grand public, utilisateurs authentifiés, back-office ?	Un moteur ouvert au public est un outil de surveillance de masse. Détermine le régime juridique et l'exigence d'audit.
4	Consentement des personnes photographiées ?	Voir §3. Sans base légale, le projet ne peut pas être mis en production.
5	Fraîcheur exigée : un nouvel événement doit-il être cherchable en secondes ou en heures ?	Temps réel → workers permanents. Différé → batch nocturne, coût GPU divisé par ~5.
6	Qualité des images : photos de foule, faible lumière, résolution ?	Détermine le seuil de qualité minimal, le taux de détection réel, et donc l'attente à fixer au métier.
7	Tolérance aux erreurs : préfère-t-on rater un événement (faux négatif) ou en proposer un faux (faux positif) ?	Fixe le seuil de décision. Deux systèmes très différents en découlent (voir §10).

Point d'attention n°2 — le plus sous-estimé. Si aucun référentiel d'identités n'existe, le système ne peut pas dire « c'est M. X ». Il ne peut que dire « la personne de votre photo apparaît dans ces événements ». C'est suffisant pour le besoin exprimé, mais cela signifie qu'**aucune galerie ne peut être pré-construite** : chaque recherche est une comparaison à froid contre l'ensemble de l'index. L'architecture ci-dessous est conçue pour ce cas, le plus général.

3. Enjeu bloquant : conformité biométrique **CRITIQUE**

Un *embedding* facial est une **donnée biométrique** permettant l'identification unique d'une personne physique. Sous RGPD (art. 9) c'est une catégorie particulière de données, dont le traitement est **interdit par principe**, sauf exception explicite. La loi malgache n° 2014-038 sur la protection des données personnelles retient une logique analogue et impose une déclaration/autorisation préalable auprès de la CMIL.

Conséquence directe : **la faisabilité technique n'est pas la contrainte dimensionnante du projet**. Les obligations suivantes doivent être traitées comme des exigences fonctionnelles de premier rang, pas comme de la documentation.

Obligation	Traduction en exigence technique
Base légale explicite	Consentement libre, spécifique et révocable des personnes photographiées, ou intérêt légitime dûment démontré. À enregistrer et horodater par personne.
Droit à l'effacement	Suppression d'une identité doit purger : embeddings, vignettes, index ANN, caches, sauvegardes. Prévoir une procédure de purge traversante et testée, pas un simple DELETE .
Minimisation	Ne jamais stocker les visages recadrés en clair au-delà du strict nécessaire ; ne conserver que le vecteur et la bounding box. Le vecteur seul ne permet pas de reconstruire un visage exploitable.
Limitation de conservation	TTL explicite sur les embeddings (ex. 24 mois) avec job de purge automatique et journal d'exécution.
Traçabilité	Journal d'audit inaltérable de toute recherche : qui, quand, image requête (empreinte cryptographique, pas l'image), résultats retournés. Non négociable — c'est la principale défense en cas de contrôle.
Sécurité (art. 32)	Chiffrement au repos de la colonne d'embeddings, chiffrement en transit, cloisonnement réseau du service d'inférence, principe du moindre privilège sur la base.
AIPD / DPIA	Analyse d'impact obligatoire (traitement biométrique à grande échelle). À produire avant le premier traitement de données réelles.

Recommandation CTO. Ouvrir le Lot 0 (cadrage juridique) immédiatement et en parallèle du Lot 1, mais **interdire tout traitement de données de production avant sa clôture**. Le développement du Lot 1 se fait sur un jeu de données public et licencié (LFW, ou images synthétiques). Ce séquençement protège le projet sans retarder l'ingénierie.

4. Principes d'architecture

Principe	Justification
P1 — Séparer écriture et lecture	Profils de charge opposés. L'indexation ne doit jamais dégrader la latence de recherche. Découplage par file de messages.
P2 — L'inférence est un service isolé	Les dépendances ML (ONNX, CUDA, OpenCV) sont lourdes et instables. Les confiner dans un conteneur dédié protège l'API métier et permet de faire évoluer le modèle indépendamment.
P3 — Le modèle est versionné comme du code	Changer de modèle d'embedding invalide tout l'index (les espaces vectoriels ne sont pas compatibles). Une colonne <code>model_version</code> et une procédure de réindexation sans coupure sont obligatoires dès la v1.
P4 — Idempotence de bout en bout	Tout traitement d'image doit pouvoir être rejoué sans dupliquer de vecteur. Clé d'idempotence = <code>(image_id, model_version)</code> .
P5 — Le système propose, l'humain décide	Aucune décision automatique irréversible sur la base d'un score. Les résultats sont des candidats classés, avec un score de confiance exposé.
P6 — Portabilité du moteur vectoriel	Accès à l'ANN via une interface applicative unique (<i>repository</i>), pour rendre la migration pgvector → Qdrant mécanique.

5. Architecture cible (vue composants)

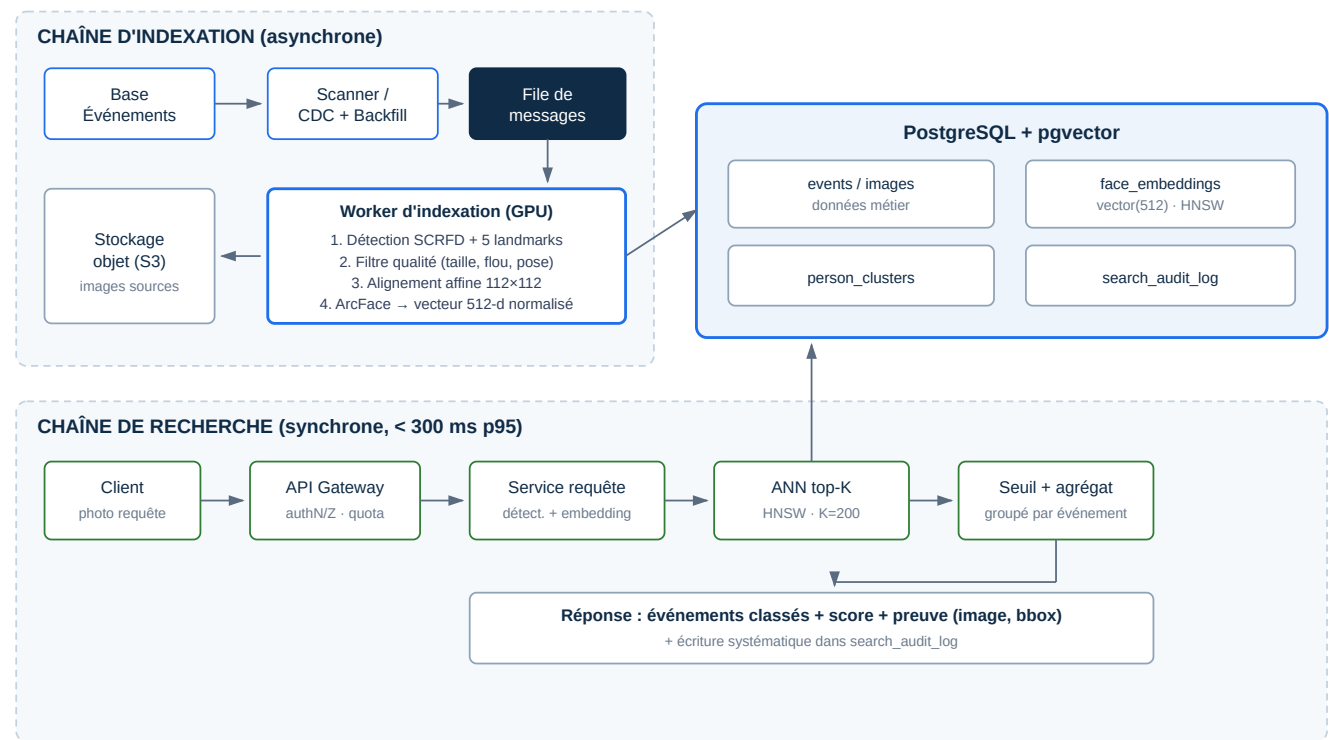


Figure 1 — Architecture cible. Les deux chaînes ne partagent que la base ; elles sont dimensionnées et déployées indépendamment.

Description des composants

Composant	Techno	Rôle & points de vigilance
Scanner / CDC	Job planifié ou <code>LISTEN/NOTIFY</code>	Détecte les images non indexées pour le <code>model_version</code> courant et publie un message. Le mode backfill et le mode incrémental utilisent le même producteur — seule la requête de sélection change.
File de messages	Redis Streams / RabbitMQ	Découplage et amortissement des pics. Impératif : <i>consumer groups</i> , ACK explicite, dead-letter queue pour les images corrompues (elles existeront).
Worker d'indexation	Python, ONNX Runtime, GPU	Scalabilité horizontale pure. Traiter les images par lots (batch 16–32) — c'est le seul levier de performance réellement significatif sur GPU.
Service de requête	FastAPI + ONNX Runtime	Même code d'inférence que le worker (bibliothèque partagée) — toute divergence entre le pré-traitement d'indexation et de requête détruit silencieusement la précision. Piège classique n°1.
Moteur ANN	pgvector, index HNSW	Transactionnel avec les données métier : pas de désynchronisation possible entre un événement supprimé et ses vecteurs.
Stockage objet	S3 / MinIO	Images sources. La base ne stocke que des références. Chiffrement au repos activé.

6. Pipeline d'indexation (détail)



Figure 2 — Étapes du traitement d'une image. Chaque étape est instrumentée (compteur + histogramme de latence).

6.1 Points de conception critiques

Normalisation L2 systématique

Les embeddings ArcFace doivent être normalisés à la norme unitaire **avant stockage**. La similarité cosinus devient alors un simple produit scalaire, ce qui autorise l'usage d'index optimisés et rend les scores directement comparables entre requêtes. Ne jamais mélanger vecteurs normalisés et non normalisés dans le même index.

Filtrage qualité en amont

C'est le levier de précision le plus rentable du système. Un visage de 20 px ou fortement flou produit un embedding bruité qui génère des faux positifs sur **toutes** les recherches futures. Règles proposées :

Critère	Seuil rejet	Justification
Taille du visage	< 40 px de côté	Sous ce seuil, l'information discriminante est perdue.
Score de détection	< 0,80	Élimine les faux visages (motifs, affiches, reflets).
Netteté (variance du Laplacien)	< 60	Flou de bougé, fréquent en événementiel nocturne.
Pose (yaw estimé)	> 45°	Profil marqué : dégradation forte de la précision d'ArcFace.

Les visages rejetés ne sont pas ignorés silencieusement : ils sont persistés avec un statut `rejected` et le motif, afin de mesurer le taux de couverture réel et d'ajuster les seuils sur données de production.

Idempotence et versionnement

La clé d'unicité est `(image_id, model_version, face_index)`. Un rejeu de message ne crée pas de doublon. Lors d'un changement de modèle, les nouveaux vecteurs sont écrits en parallèle des anciens avec un `model_version` incrémenté, puis la bascule de lecture se fait par changement d'une variable de configuration — **réindexation sans coupure de service**.

Regroupement en identités (clustering)

Optionnel en v1, recommandé en v2. Un clustering incrémental (type *HDBSCAN* ou agglomératif par seuil) regroupe les visages en identités latentes. Bénéfices :

- La recherche peut d'abord frapper les **centroïdes de clusters** (quelques dizaines de milliers) plutôt que tous les vecteurs — accélération d'un ordre de grandeur.
- Le vote par cluster réduit fortement les faux positifs : une personne présente sur 5 photos d'un événement est confirmée 5 fois.
- Le droit à l'effacement s'exerce au niveau du cluster, pas visage par visage.

Robustesse opérationnelle

- **Reprise sur erreur** : 3 tentatives avec backoff exponentiel, puis mise en DLQ avec le motif. Une image corrompue ne doit jamais bloquer un lot.

- **Backfill contrôlé** : le traitement de l'historique s'exécute avec une priorité de file inférieure au flux incrémental, afin de ne pas retarder les nouveaux événements.
- **Progression observable** : métrique `images_indexed / images_total` par `model_version`, exposée en tableau de bord.

7. Pipeline de recherche (détail)

La recherche est une opération en cinq temps. La partie non triviale n'est pas la recherche vectorielle — elle est résolue — mais le **passage du visage à l'événement**.

7.1 Séquence

1. **Extraction requête** — détection sur l'image soumise. Si plusieurs visages sont présents, exiger que l'appelant désigne lequel (ou retenir le plus grand/centré, avec retour explicite du choix dans la réponse). Ne jamais deviner silencieusement.
2. **Recherche ANN** — top-K sur l'index HNSW, avec K large (150–300). K doit être > au nombre de visages attendus pour une même personne, sinon les événements les plus riches écrasent les autres.
3. **Seuillage** — élimination des candidats sous le seuil de similarité calibré (§10).
4. **Agrégation par événement** — regroupement des visages retenus par `event_id`, avec un score d'événement.
5. **Restitution** — événements classés, chacun accompagné de sa *preuve* : l'image et la bounding box du meilleur visage correspondant, pour permettre une vérification humaine immédiate.

7.2 Score d'événement DÉCISION CLÉ

Le choix de la fonction d'agrégation détermine le comportement perçu du produit :

Fonction	Comportement	Verdict
<code>max(similarité)</code>	Un seul bon visage suffit à classer l'événement en tête.	Sensible, mais un unique faux positif remonte un événement erroné au sommet. RISQUÉ SEUL
<code>moyenne</code>	Dilue le signal : un événement avec 1 bonne correspondance sur 30 visages est pénalisé.	Inadapté. NON
<code>max + bonus de corroboration</code>	<code>score = max_sim + λ · log(1 + n_matches)</code> , $\lambda \approx 0,02$.	Recommandé. Récompense la répétition sans écraser le signal principal. RETENU

7.3 Optimisations

- **Filtrage pré-ANN** : si l'utilisateur restreint par date ou par lieu, appliquer le filtre **avant** la recherche vectorielle. pgvector le gère nativement via des index partiels ; c'est un gain massif sur de gros volumes.
- **Cache de requête** : clé = hachage perceptuel de l'image + paramètres. Une même photo est fréquemment resoumise.
- **Ré-ordonnement exact** : l'ANN retourne des candidats approchés ; recalculer la similarité exacte sur les K candidats (coût négligeable) avant seuillage, pour éliminer l'erreur d'approximation à la frontière de décision.

8. Modèle de données

```
-- Extension vectorielle
CREATE EXTENSION IF NOT EXISTS vector;

-- Existant (rappel) : events(id, description, event_date, address, ...)

CREATE TABLE event_images (
  id          BIGSERIAL PRIMARY KEY,
  event_id    BIGINT NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  storage_uri TEXT NOT NULL,          -- s3://bucket/key
  content_hash CHAR(64) NOT NULL,     -- SHA-256, déduplication
  width       INT, height INT,
  indexed_at  TIMESTAMPTZ,
  index_status TEXT NOT NULL DEFAULT 'pending', -- pending|done|failed|skipped
  UNIQUE (content_hash)
);

CREATE TABLE face_embeddings (
  id          BIGSERIAL PRIMARY KEY,
  image_id    BIGINT NOT NULL REFERENCES event_images(id) ON DELETE CASCADE,
  event_id    BIGINT NOT NULL REFERENCES events(id) ON DELETE CASCADE, --
  dénormalisé : évite une jointure sur le chemin chaud
  face_index  SMALLINT NOT NULL,     -- n-ième visage de l'image
  bbox        INT[4] NOT NULL,       -- x, y, w, h
  det_score   REAL NOT NULL,
  quality_score REAL NOT NULL,
  embedding   vector(512) NOT NULL,  -- normalisé L2
  model_version TEXT NOT NULL,       -- ex. 'arcface-r100-v1'
  cluster_id  BIGINT REFERENCES person_clusters(id),
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (image_id, model_version, face_index)
);

-- Index ANN : HNSW, distance cosinus.
-- m = connexions/nœud (mémoire vs rappel) ; ef_construction = qualité de construction.
CREATE INDEX idx_face_hnsw ON face_embeddings
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64)
  WHERE model_version = 'arcface-r100-v1'; -- index partiel : isole la version active

CREATE INDEX idx_face_event ON face_embeddings (event_id);

CREATE TABLE person_clusters (
  id          BIGSERIAL PRIMARY KEY,
  centroid    vector(512) NOT NULL,
  face_count  INT NOT NULL DEFAULT 0,
  consent_state TEXT NOT NULL DEFAULT 'unknown', -- granted|withdrawn|unknown
  purge_after TIMESTAMPTZ -- TTL de conservation
);

-- Journal d'audit : exigence de conformité, non optionnel
CREATE TABLE search_audit_log (
  id          BIGSERIAL PRIMARY KEY,
  actor_id    TEXT NOT NULL,
  query_hash  CHAR(64) NOT NULL, -- empreinte de l'image requête, JAMAIS l'image
  result_count INT NOT NULL,
  top_score   REAL,
```

```
threshold_used REAL NOT NULL,  
model_version TEXT NOT NULL,  
created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Choix notable — dénormalisation de `event_id` dans `face_embeddings`. Elle évite une jointure sur le chemin de lecture le plus sollicité et permet de filtrer par événement directement dans la requête ANN. La cohérence est garantie par le `ON DELETE CASCADE` et par le fait que la relation image→événement est immuable.

Requête de recherche type

```
WITH candidates AS (  
  SELECT event_id, image_id, bbox,  
         1 - (embedding <=> $1::vector) AS similarity  
  FROM   face_embeddings  
 WHERE  model_version = $2  
        AND quality_score >= 0.5  
 ORDER  BY embedding <=> $1::vector      -- utilise l'index HNSW  
 LIMIT  200  
)  
SELECT event_id,  
       MAX(similarity) + 0.02 * ln(1 + COUNT(*)) AS event_score,  
       COUNT(*) AS match_count  
FROM   candidates  
WHERE  similarity >= $3                  -- seuil calibré  
GROUP  BY event_id  
ORDER  BY event_score DESC  
LIMIT  50;
```

9. Choix technologiques & alternatives

9.1 Modèles de vision

Étage	Retenu	Alternatives	Justification
Détection	SCRFD (InsightFace)	RetinaFace (+ précis, – rapide), YOLOv8-face, MTCNN (obsolète)	Meilleur compromis précision/vitesse sur visages petits et nombreux — le cas exact des photos d'événement.
Embedding	ArcFace R100 (buffalo_l, 512-d)	AdaFace (meilleur sur basse qualité), FaceNet (dépassé), CosFace	Référence du domaine, licence permissive, écosystème ONNX mature, précision > 99,8 % sur LFW. AdaFace mérite un benchmark comparatif si le corpus contient beaucoup d'images dégradées.
Exécution	ONNX Runtime	TensorRT (+ rapide, – portable), PyTorch (– efficace en inférence)	Portabilité CPU/GPU sans réécriture, dépendances légères, chargement rapide.

9.2 Moteur de recherche vectorielle

Option	Forces	Limites	Verdict
pgvector	Une seule base ; transactions ACID avec le métier ; filtrage SQL riche ; exploitation déjà maîtrisée ; sauvegardes unifiées.	Dégradation au-delà de ~10 M vecteurs ; construction d'index HNSW coûteuse en RAM ; pas de sharding natif.	RETENU V1 Couvre largement l'hypothèse de 3 M vecteurs.
Qdrant	Excellent filtrage + ANN combiné ; quantification scalaire/binaire ; horizontalement scalable.	Second système à opérer, synchroniser et sauvegarder ; cohérence éventuelle avec la base métier.	PLAN B Bascule si > 10 M vecteurs ou latence dégradée.
Milvus	Conçu pour le milliard de vecteurs ; multiples types d'index.	Complexité opérationnelle importante (etcd, Pulsar, MinIO).	Surdimensionné ici. NON
FAISS (en process)	Le plus rapide en brut ; aucun serveur.	Pas de persistance ni de mise à jour incrémentale simple ; index en RAM à reconstruire.	Utile comme référence de <i>benchmark</i> , pas en production.

9.3 Choix d'index HNSW vs IVFFlat

HNSW est recommandé : rappel supérieur à latence égale, et surtout **il accepte les insertions incrémentales** sans reconstruction — indispensable pour un flux d'indexation continu. IVFFlat exige un jeu représentatif pour l'entraînement des centroïdes et se dégrade quand la distribution évolue. Le surcoût mémoire de HNSW ($\approx m \times 8$ octets par vecteur, soit ~380 Mo pour 3 M vecteurs à $m=16$, en sus des 6 Go de vecteurs) est acceptable.

10. Qualité : seuils, métriques, biais

Le seuil de similarité est le paramètre produit le plus important du système. Il ne se choisit pas en lisant un article : il se **calibre sur les données réelles du client**. Un seuil transposé d'un benchmark académique donnera systématiquement de mauvais résultats en production.

10.1 Arbitrage faux positifs / faux négatifs

Réglage	Seuil cosinus	Conséquence	Usage adapté
Permissif	$\approx 0,30$	Peu d'oublis, mais des événements erronés remontent.	Exploration assistée, l'utilisateur trie visuellement.
Équilibré	$\approx 0,38$	Compromis par défaut, à valider par mesure.	Recommandé comme point de départ.
Strict	$\approx 0,50$	Quasi aucun faux positif, mais des présences réelles manquées.	Tout contexte à conséquence pour la personne.

10.2 Protocole de calibration **OBLIGATOIRE**

- Constituer un jeu de validation annoté de **500 à 1 000 paires** issues du corpus réel (mêmes conditions de prise de vue, même population).
- Tracer la courbe DET (FAR vs FRR) sur l'ensemble des seuils.
- Fixer d'abord le **FAR cible** (ex. 1 faux positif pour 10 000 comparaisons), puis en déduire le seuil et constater le FRR obtenu — jamais l'inverse.
- Rejouer cette calibration à chaque changement de modèle et au moins semestriellement.

10.3 Métriques à instrumenter en continu

Métrique	Ce qu'elle révèle
Taux de détection par image	Qualité du corpus ; une chute signale un problème d'ingestion en amont.
Distribution des scores de qualité	Dérive du corpus (nouvel appareil photo, nouvelles conditions).
Distribution des similarités top-1	Un pic proche du seuil indique un seuil mal placé.
Taux de clic sur les résultats par rang	Meilleur signal de pertinence réelle disponible sans annotation manuelle.
Recherches à zéro résultat	Seuil trop strict, ou couverture d'indexation insuffisante.

10.4 Biais démographiques **RESPONSABILITÉ**

Les modèles de reconnaissance faciale présentent des écarts de performance documentés selon la couleur de peau, le genre et l'âge (cf. NIST FRVT). Un seuil unique appliqué à toute la population produit mécaniquement un taux d'erreur plus élevé sur certains groupes. Deux exigences en découlent :

- **Mesurer** le FAR/FRR par sous-groupe sur le jeu de validation, et publier ces écarts dans la documentation du système.
- **Ne jamais présenter un résultat comme une certitude.** L'interface doit exposer le score et l'image-preuve, et le vocabulaire doit rester probabiliste (« correspondance probable », jamais « identifié »).

11. Dimensionnement & performance

Sur l'hypothèse de travail : **100 000 événements × 8 images × 4 visages ≈ 3,2 M vecteurs**.

Poste	Estimation	Détail
Volume vecteurs	~6,6 Go	3,2 M × 512 × 4 octets (float32).
Surcoût index HNSW	~0,4 Go	m = 16. L'index doit tenir en RAM : prévoir 16 Go de RAM minimum sur l'instance PostgreSQL.
Débit d'indexation (GPU T4)	~40 img/s	Détection + embedding, batch 32. Backfill complet ≈ 5,5 heures pour 800 k images avec 1 worker.
Débit d'indexation (CPU 8 cœurs)	~3 img/s	Backfill ≈ 3 jours. Acceptable pour une reprise unique, insuffisant pour un flux soutenu.
Latence de recherche	~150–300 ms p95	Décomposition : détection+embedding requête ~80 ms (GPU) / ~250 ms (CPU) ; ANN top-200 ~15 ms ; agrégation ~5 ms.
Débit de recherche	~50 req/s par nœud	Le goulot est l'inférence de la requête, pas l'ANN. Scalabilité horizontale du service de requête.

Levier de dimensionnement. Le GPU n'est nécessaire que pour le backfill initial et les pics d'indexation. Architecture économique recommandée : **workers d'indexation GPU éphémères** (démarrés à la demande, arrêtés à vide) + **service de requête CPU permanent**. Si la latence CPU de 250 ms sur la requête devient gênante, un seul petit GPU partagé suffit à la desservir.

Quand l'architecture devra évoluer

Signal	Action
> 10 M vecteurs, ou latence ANN p95 > 100 ms	Migration vers Qdrant, ou quantification scalaire (int8) de pgvector : -75 % de mémoire pour ~1 % de rappel.
Backfill > 24 h	Parallélisation multi-workers GPU (partitionnement par plage d' <code>image_id</code>).
> 200 recherches/s	Cache d'embeddings de requête + montée en nombre de répliques du service de requête.

12. Contrat d'API

POST /api/v1/search/by-face

Content-Type: multipart/form-data

Authorization: Bearer <token>

image : fichier (JPEG/PNG, ≤ 10 Mo)
face_index : entier (optionnel – si l'image requête contient plusieurs visages)
threshold : réel (optionnel, défaut = seuil calibré du système)
date_from : date (optionnel – filtre pré-ANN)
date_to : date (optionnel)
limit : entier (défaut 20, max 100)

200 OK

```
{
  "query": {
    "faces_detected": 2,
    "face_used": { "index": 0, "bbox": [120,80,64,64], "quality": 0.91 },
    "model_version": "arcface-r100-v1"
  },
  "threshold_used": 0.38,
  "results": [
    {
      "event_id": 4821,
      "description": "Séminaire annuel",
      "event_date": "2026-03-14",
      "address": "Antananarivo, Ivandry",
      "confidence": 0.78,           // score d'événement normalisé
      "match_count": 3,           // corroboration
      "evidence": {               // preuve pour vérification humaine
        "image_id": 99312,
        "image_url": "https://.../signed?exp=...",
        "bbox": [412, 190, 78, 78],
        "similarity": 0.71
      }
    }
  ]
}
```

409 Conflict – aucun visage détecté, ou plusieurs sans face_index précisé

413 Too Large – image dépassant la limite

429 Too Many – quota de recherche dépassé (protection anti-surveillance)

Principes de conception de l'API

- **Toujours retourner la preuve.** Un résultat sans image ni bbox est invérifiable et donc inutilisable de façon responsable.
- **Exposer le seuil utilisé et la version du modèle.** Sans cela, un résultat n'est pas reproductible ni auditable.
- **409 plutôt que devinette** en cas d'ambiguïté sur le visage requête.
- **Quota strict par acteur.** Un accès sans limite de débit transforme le service en outil de profilage de masse ; le quota est une mesure de conformité, pas seulement de performance.
- **URLs d'images signées et à durée de vie courte,** jamais d'accès direct au stockage objet.

13. Sécurité & exploitation

Domaine	Mesures
Chiffrement	TLS en transit ; chiffrement au repos du volume PostgreSQL et du bucket objet. Étudier le chiffrement applicatif de la colonne <code>embedding</code> — attention : incompatible avec l'index ANN, donc réservé aux colonnes non indexées.
Cloisonnement	Le service d'inférence n'est joignable que depuis le réseau interne. Aucune exposition publique directe. Le worker d'indexation n'a pas d'accès en lecture au journal d'audit.
Moindre privilège	Trois rôles base distincts : <code>indexer</code> (INSERT sur embeddings), <code>searcher</code> (SELECT seul + INSERT audit), <code>admin</code> . Aucun service ne dispose de DELETE hors procédure de purge dédiée.
Purge RGD	Procédure unique et testée : suppression du cluster, des embeddings, des entrées de cache, invalidation des URLs signées, et marquage pour exclusion des restaurations de sauvegarde . Exécution journalisée.
Surface d'attaque ML	Valider strictement les images entrantes (type MIME réel, dimensions, bombes de décompression). Les bibliothèques de décodage d'images sont un vecteur d'exécution de code historique.
Observabilité	Traces distribuées sur les deux chaînes, métriques Prometheus (latence par étage, taille de file, profondeur DLQ, taux de rejet qualité), alertes sur file croissante et sur chute du taux de détection.
Reprise d'activité	Les embeddings sont reconstructibles à partir des images sources. La priorité de sauvegarde porte donc sur le stockage objet et les données métier ; l'index peut être régénéré (coût = durée de backfill).

14. Coûts & build vs buy

Option	Coût indicatif	Avantages	Inconvénients rédhibitoires
Auto-hébergé (recommandé)	Infra seule : ~150–400 €/mois (1 VM applicative + 1 base + GPU à la demande)	Coût marginal nul par requête ; souveraineté totale des données biométriques ; contrôle du seuil et du modèle.	Charge d'ingénierie initiale et de maintenance du modèle.
AWS Rekognition	~1 \$/1 000 images indexées + 1 \$/1 000 recherches	Mise en œuvre en jours ; pas de MLOps.	Transfert de données biométriques hors juridiction — difficilement compatible avec l'exigence de conformité. Coût linéaire au volume. Verrouillage fournisseur (les vecteurs ne sont pas exportables).
Azure Face API	Comparable	Idem.	Idem, et l'accès aux fonctions d'identification est soumis à validation de cas d'usage par Microsoft.

Arbitrage. À partir d'environ 200 000 images indexées, l'auto-hébergement devient moins coûteux que le cloud managé. Mais l'argument décisif n'est pas financier : **la nature biométrique des données impose la maîtrise du lieu et du cycle de vie du traitement.** Le verrouillage fournisseur est également plus grave qu'ailleurs, puisqu'un changement de prestataire impose une réindexation totale du corpus.

15. Trajectoire de mise en œuvre

Lot	Livrable	Durée	Critère de sortie
0	Cadrage juridique : base légale, DPIA, politique de rétention, matrice d'habilitation	2 sem. BLOQUANT	DPIA validée ; mécanisme de consentement défini. Prérequis à tout traitement de données réelles.
1	Socle d'indexation : schéma, workers, détection+embedding, file, backfill, reprise sur erreur	4–5 sem.	Corpus complet indexé ; taux de couverture mesuré et > 90 % des images traitées sans erreur.
2	Recherche : index HNSW, API, agrégation, calibration du seuil sur jeu annoté	3 sem.	p95 < 300 ms ; FAR mesuré < 10^{-4} au seuil retenu ; courbe DET documentée.
3	Industrialisation : observabilité, purge RGPD testée, journal d'audit, back-office	3–4 sem.	Purge d'une identité vérifiée de bout en bout ; tableaux de bord en place ; runbook rédigé.
4	<i>Optionnel</i> — clustering d'identités, recherche multi-visages, réindexation sans coupure	3 sem.	Gain de précision mesuré vs baseline du lot 2.

Preuve de concept préalable — 5 jours **RECOMMANDÉ**

Avant d'engager le Lot 1, valider les hypothèses sur **1 000 images réelles du client** : mesurer le taux de détection effectif, la distribution des scores de qualité, et la séparabilité des scores sur 100 paires annotées. Cette semaine détermine si les seuils proposés tiennent sur le corpus réel — et si le besoin est réalisable au niveau de qualité attendu par le métier. **C'est l'investissement au meilleur rapport risque/coût du projet.**

16. Registre des risques

Risque	Prob.	Impact	Mitigation
Absence de base légale pour le traitement biométrique	Élevée	Critique	Lot 0 bloquant ; développement sur données publiques jusqu'à validation ; DPIA formalisée.
Qualité du corpus insuffisante (photos de foule, basse lumière)	Élevée	Élevé	PoC de 5 jours sur données réelles ; filtre qualité explicite ; attentes métier calibrées sur mesures, pas sur benchmarks.
Faux positifs perçus comme des affirmations d'identité	Moyenne	Élevé	Vocabulaire probabiliste imposé dans l'UI ; image-preuve systématique ; validation humaine avant toute action.
Divergence pré-traitement indexation / requête	Moyenne	Élevé	Bibliothèque d'inférence unique et partagée ; test de non-régression comparant les deux chemins sur une image de référence.
Changement de modèle invalidant l'index	Moyenne	Moyen	<code>model_version</code> dès la v1 ; index partiel par version ; procédure de double écriture puis bascule.
Dégradation de pgvector à la montée en volume	Faible	Moyen	Interface <i>repository</i> abstraite ; alerte sur latence ANN ; plan de bascule Qdrant documenté.
Détournement en outil de surveillance	Moyenne	Critique	Quota par acteur ; journal d'audit inaltérable ; habilitations restreintes ; revue périodique des usages.
Biais démographique non mesuré	Élevée	Élevé	Évaluation FAR/FRR par sous-groupe ; publication des écarts ; seuils différenciés si l'écart est significatif.

17. Décisions d'architecture (ADR)

N°	Décision	Motif
001	Séparer les chaînes d'indexation et de recherche, découplées par file de messages	Profils de charge opposés ; l'indexation ne doit jamais dégrader la latence de recherche ; scalabilité indépendante.
002	PostgreSQL + pgvector comme moteur vectoriel en v1	Un seul système à opérer ; transactions ACID avec le métier ; suffisant jusqu'à ~10 M vecteurs. Bascule Qdrant prévue par abstraction.
003	ArcFace 512-d normalisé L2, exécuté en ONNX Runtime	État de l'art, licence permissive, portabilité CPU/GPU sans réécriture.
004	Index HNSW plutôt qu'IVFFlat	Insertions incrémentales sans reconstruction ; rappel supérieur à latence égale.
005	<code>model_version</code> obligatoire, index partiel par version	Rend la réindexation possible sans coupure de service — sinon tout changement de modèle impose un arrêt.
006	Agrégation par événement : $\max + \lambda \cdot \log(1+n)$	Conserve la sensibilité du max tout en récompensant la corroboration multi-visages.
007	Ne pas stocker les visages recadrés ; conserver vecteur + bbox uniquement	Minimisation des données ; réduit fortement l'impact d'une compromission.
008	Journal d'audit de toute recherche, non désactivable	Exigence de conformité et principale défense en cas de contrôle ou de contestation.
009	Auto-hébergement plutôt que service cloud de reconnaissance faciale	Souveraineté sur données biométriques ; coût marginal nul ; pas de verrouillage imposant une réindexation totale.
010	Aucune décision automatique : le système propose des candidats scorés avec preuve	Les taux d'erreur et les biais connus interdisent de traiter un score comme une identification.

Conclusion

Le besoin est **techniquement réalisable avec des briques matures** et une équipe restreinte, en un trimestre. Les modèles de reconnaissance faciale sont aujourd'hui des commodités ; la valeur d'ingénierie se situe ailleurs :

- **Dans la conformité** — c'est le seul risque capable d'arrêter le projet, et il doit être traité en premier, en parallèle mais en amont de toute donnée réelle.
- **Dans la calibration** — un seuil mal placé produit un système qui « fonctionne » techniquement mais est inutilisable pour le métier. La preuve de concept de 5 jours sur données réelles est l'investissement prioritaire.
- **Dans la discipline d'ingénierie** — versionnement du modèle, idempotence, unicité du chemin d'inférence, journal d'audit. Ces quatre points coûtent peu s'ils sont posés dès la v1, et très cher s'ils sont ajoutés après.

Prochaine étape immédiate. Obtenir les réponses aux sept questions du §2.2 — en particulier la volumétrie, l'existence d'un référentiel d'identités et le profil des utilisateurs autorisés à rechercher. Ces réponses confirmeront ou ajusteront le dimensionnement retenu ici. En parallèle, lancer le Lot 0 et la preuve de concept sur 1 000 images réelles.